



{riviera}
dev

LANGCHAIN4J-CDI @ RIVIERA DEV



LANGCHAIN4J-CDI

IA GÉNÉRATIVE
NATIVEMENT INJECTABLE DANS JAKARTA EE

Yann Blazart, Emmanuel Hugonnet & Antoine Sabot-Durand

RivieraDev 2026

QUI SUIS-JE ?



YANN BLAZART

25+ ans en Java • Passionné de CDI • Co-créateur de langchain4j-cdi
Speaker pendant mon temps libre (et quand on veut bien de moi)

Jakarta EE

CDI

LangChain4j

Tech Lead @ **SCIAM** — Conseil et expertise IT

@yblazart

MON CO-PILOTE



EMMANUEL HUGONNET (EHSOVOIE)

25+ ans en Java • Développeur • Co-créateur de langchain4j-cdi
Développeur cœur WildFly

WildFly

Apache NetBeans

LangChain4j

A2A Java SDK



Celui qui fait tourner **WildFly** pour que vous n'ayez pas à le faire

@ehsavoie

LE CONTEXTE AUJOURD'HUI

Vous avez peut-être vu :

LangChain4j +  Quarkus =  Chatbot Intelligent

Mais en production, vous avez :

 WildFly

 Payara

 Liberty

 Helidon

 GlassFish

 Quarkus

Alors on fait quoi ? 

LA RÉPONSE :

@RegisterAIService

CDI fait le pont entre **LangChain4j** et votre serveur

"Injecter un agent IA aussi naturellement qu'un EntityManager"

PROGRAMME D'AUJOURD'HUI



Architecture — Comment ça fonctionne sous le capot



@RegisterAIService — Injection en action



Fonctionnalités progressives — Memory, RAG, Tools, MCP



Prêt pour la production — Fault Tolerance & Telemetry



Guardrails — Valider l'entrée et la sortie de vos agents



A2A : Agent-to-Agent — Orchestration multi-agents via le protocole A2A

LE PROBLÈME

```
// Avec LangChain4j "vanille"... c'est verbeux
ChatModel model = OpenAiChatModel.builder()
    .apiKey(System.getenv("OPENAI_API_KEY"))
    .modelName("gpt-4o")
    .build();

ChatMemory memory = MessageWindowChatMemory.builder()
    .maxMessages(20)
    .build();

Assistant assistant = AiServices.builder(Assistant.class)
    .chatLanguageModel(model)
    .chatMemory(memory)
    .tools(new BookingTools())
    .build();
```

Pas très... enterprise, non ? 🤖

LA SOLUTION CDI

```
@RegisterAIService(chatModelName = "my-model",  
    tools = BookingTools.class)  
public interface Assistant {  
  
    @SystemMessage("You are a booking assistant")  
    String chat(@UserMessage String message);  
}
```

```
// Et dans votre code métier :  
@Inject  
Assistant assistant;  
  
// C'est tout. Vraiment.  
String response = assistant.chat("Book me a table");
```

SPRING AI VS LANGCHAIN4J-CDI

Même puissance. **ADN différent.**

♣ SPRING AI / SPRING BOOT 3.X

```
// Configuration bean
@Configuration
public class AssistantConfig {
    @Bean
    ChatClient chatClient(
        ChatClient.Builder b,
        VectorStore vs, ChatMemory mem) {
        return b.defaultSystem("You are helpful")
            .defaultAdvisors(
                new QuestionAnswerAdvisor(vs),
                new MessageChatMemoryAdvisor(mem))
            .defaultTools(new WeatherTools())
            .build();
    }
}

// Service that uses it
@Service
public class AssistantService {
    private final ChatClient client;
    String chat(String user, String msg) {
        return client.prompt().user(msg)
            .advisors(a -> a.param(CONV_ID, user))
            .call().content();
    }
}
```

~25 lignes · 2 classes · 1 builder chain

◆ LANGCHAIN4J-CDI / JAKARTA EE

```
// That's it. One interface.
@RegisterAIService(
    chatMemoryProviderSupplier
        = MemoryProvider.class,
    tools = WeatherTools.class,
    retrievalAugmentor = RagAugmentor.class)
public interface Assistant {

    @SystemMessage("You are helpful")
    String chat(@MemoryId String user,
        @UserMessage String msg);
}

// Inject anywhere. Done.
@Inject
Assistant assistant;
assistant.chat("yann", "Bonjour !");
```

~10 lignes · 1 interface · 0 passe-partout

Déclaratif. Injectable. Portable. — Juste CDI qui fait ce que CDI fait de mieux.



L'INJECTION DE DÉPENDANCES C'EST COMME LA MÉDECINE :

Bien faite, ça ne fait pas mal.

Mal faite, vous avez une `NullPointerException`.

Et avec CDI 4.1, on a même l'injection... *à la compilation*.

CDI 4.1 : BUILD COMPATIBLE EXTENSIONS

PORTABLE EXTENSION

Traitement au runtime

WildFly, Payara, GlassFish, Liberty

```
void processAnnotatedType(
    @Observes
    ProcessAnnotatedType<?> pat) {
    // Scan au runtime
}
```

BUILD COMPATIBLE EXT.

Traitement à la compilation

Quarkus, Helidon

```
@Enhancement(
    types = RegisterAIService.class,
    withAnnotations = RegisterAIService.class)
void enhance(ClassConfig cfg) {
    // Enrichissement au build
}
```

langchain4j-cdi fournit les deux — c'est *transparent* pour vous 🎉

Oui, ça tourne aussi sur **Quarkus**. Ils peuvent mettre leur extension à la retraite. 😊

SOUS LE CAPOT

1. L'extension scanne pour `@RegisterAIService`
2. Pour chaque interface trouvée, elle lit les attributs :
`chatModelName`, `tools`, `chatMemoryProviderName`,
`retrievalAugmentorName`
3. Elle crée un **bean CDI synthétique** :
→ Proxy `LangChain4j` enrobé dans un bean CDI managé
4. Les composants sont résolus via un **SPI** (`LLMConfig`) :
`dev.langchain4j.cdi.plugin.<name>.class=...`
→ Implémentation **MicroProfile Config** fournie par `langchain4j-cdi-config`
→ Implémentation **WildFly** fournie par `wildfly-ai-feature-pack`
5. `@Inject Assistant assistant;` → ✨ Ça marche !

CONFIGURATION : MICROPROFILE CONFIG

```
# Configuration du modèle
dev.langchain4j.cdi.plugin.my-model.class=\
    dev.langchain4j.model.openai.OpenAiChatModel
dev.langchain4j.cdi.plugin.my-model.config.api-key=${OPENAI_API_KEY}
dev.langchain4j.cdi.plugin.my-model.config.model-name=gpt-4o

# Configuration de la mémoire
dev.langchain4j.cdi.plugin.my-memory.class=\
    dev.langchain4j.memory.chat.MessageWindowChatMemory
dev.langchain4j.cdi.plugin.my-memory.config.max-messages=20

# Liaison vers d'autres beans : lookup:@default, lookup:@all, lookup:name
dev.langchain4j.cdi.plugin.my-retriever.config.embeddingStore=\
    Lookup:@default
```

Pas de code, juste des properties. **On est en 2026.**

Alternative WildFly : Le subsystem AI WildFly permet aussi la configuration via standalone.xml

CONFIGURATION : SUBSYSTEM AI WILDFLY

```
←!— standalone.xml - Configuration du subsystem AI WildFly →  
<subsystem xmlns="urn:jboss:domain:ai:1.0">  
  ←!— Configuration du modèle Ollama →  
  <chat-language-models>  
    <ollama-chat-model name="ollama"  
      base-url="http://127.0.0.1:11434"  
      model-name="llama3.1:8b"  
      temperature="0.9"  
      connect-timeout="600000"  
      log-requests="true"  
      log-responses="true" />  
  </chat-language-models>  
  
  ←!— Configuration du client MCP SSE →  
  <mcp>  
    <mcp-tool-provider name="mcp" mcp-clients="mcp-sse" />  
    <mcp-client-sse name="mcp-sse"  
      socket-binding="mcp-sse"  
      sse-path="/sse"  
      connect-timeout="6000000"  
      log-requests="true"  
      log-responses="true" />  
  </mcp>  
</subsystem>  
  
←!— Socket binding pour le serveur MCP →  
<outbound-socket-binding name="mcp-sse">  
  <remote-destination host="localhost" port="8090" />  
</outbound-socket-binding>
```

Configuration WildFly native -- Zéro properties ! 🚀



DEMO 1

@REGISTERAISERVICE EN ACTION

- ✓ Chat simple avec @RegisterAIService
- ✓ Streaming : réponses en temps réel 
- ✓ Vision : analyse d'images 

On bascule sur IntelliJ...

(Si IntelliJ refuse de coopérer, on a des slides de secours 😊)

L'INTERFACE AI SERVICE

```
package com.example.demo1;

import dev.langchain4j.service.SystemMessage;
import dev.langchain4j.service.UserMessage;
import dev.langchain4j.cdi.spi.RegisterAIService;

@RegisterAIService(chatModelName = "my-model")
public interface ChatAssistant {

    @SystemMessage("""
        You are a Viking skald performing in the great hall.
        You tell jokes about warriors, longships, axes,
        and the epic chaos of Viking raids. Your humor is sharp,
        self-deprecating, and full of Norse bravado.
        """)
    String chat(@UserMessage String userMessage);
}
```

L'ENDPOINT REST

```
@Path("/chat")
@ApplicationScoped
public class ChatResource {

    @Inject
    ChatAssistant assistant; // ← Injection CDI, classique !

    @POST
    @Consumes(MediaType.TEXT_PLAIN)
    @Produces(MediaType.TEXT_PLAIN)
    public String chat(String message) {
        return assistant.chat(message);
    }
}
```

Aucune dépendance LangChain4j dans votre code métier !

STREAMING : RÉPONSES EN TEMPS RÉEL

```
@RegisterAIService(streamingChatModelName = "my-streaming-model")
public interface ChatAssistantStreaming {

    @SystemMessage("You are a creative assistant...")
    TokenStream chatStream(@UserMessage String userMessage);
}
```

```
// Dans l'endpoint JAX-RS avec Server-Sent Events
@GET
@Path("/stream")
@Produces(MediaType.SERVER_SENT_EVENTS)
public void chatStream(@QueryParam("message") String message,
                      @Context SseEventSink sink, @Context Sse sse) {
    TokenStream tokenStream = streamingAssistant.chatStream(message);

    tokenStream
        .onPartialResponse(token → sink.send(sse.newEvent("token", token)))
        .onCompleteResponse(resp → { sink.send(sse.newEvent("done", ""));
                                     sink.close(); })
        .onError(error → { sink.send(sse.newEvent("error", error.getMessage()));
                           sink.close(); })
        .start();
}
```

Effet ChatGPT : les tokens arrivent au fur et à mesure ! 🎬

VISION : ANALYSE D'IMAGES

```
@WebServlet("/uploadServlet")
@MultipartConfig
public class ImageAnalyzerServlet extends HttpServlet {

    @Inject
    @Named("vision-model")
    ChatModel visionModel; // ← Un simple ChatModel !

    @Override
    protected void doPost(HttpServletRequest request,
                          HttpServletResponse response) throws ... {
        Part file = request.getPart("file");

        UserMessage userMessage = UserMessage.from(
            TextContent.from("Describe this image in detail"),
            ImageContent.from(encodeBase64(file.getInputStream()),
                             file.getContentType())
        );

        ChatResponse answer = visionModel.chat(userMessage);
        response.getWriter().write(answer.aiMessage().text());
    }
}
```

Multimodal = texte + image dans le même message

CONFIGURATION : STREAMING & VISION

```
# Configuration du modèle streaming
dev.langchain4j.cdi.plugin.my-streaming-model.class=\
    dev.langchain4j.model.openai.OpenAiStreamingChatModel
dev.langchain4j.cdi.plugin.my-streaming-model.config.api-key=${OPENAI_API_KEY}
dev.langchain4j.cdi.plugin.my-streaming-model.config.model-name=gpt-4o

# Configuration du modèle vision (via CDI @Named)
dev.langchain4j.cdi.plugin.vision-model.class=\
    dev.langchain4j.model.ollama.OllamaChatModel
dev.langchain4j.cdi.plugin.vision-model.config.base-url=http://localhost:11434
dev.langchain4j.cdi.plugin.vision-model.config.model-name=llama3.2-vision
dev.langchain4j.cdi.plugin.vision-model.config.timeout=PT5M
```

`streamingChatModelName` pour le streaming
`@Named + ChatModel` pour l'injection directe de modèle

CE QU'ON VIENT DE VOIR



CHAT CLASSIQUE

@RegisterAIService
+ @Inject
= Skald Viking 🗡️🎤



STREAMING

TokenStream
+ Server-Sent Events
= Saga Viking en live 🎵



VISION MULTIMODALE

TextContent + ImageContent
= Analyse d'images avec l'IA

Tout ça avec CDI. Zéro code de plomberie. 🎉



UN LLM SANS MÉMOIRE...

...c'est comme un développeur le lundi matin.

Il a oublié tout ce qu'il savait vendredi.

Heureusement, on a ChatMemory !

MÉMOIRE DE CONVERSATION

```
@RegisterAIService(chatModelName = "my-model",  
                    chatMemoryProviderName = "my-memory")  
public interface ChatAssistant {  
    String chat(@MemoryId String sessionId,  
               @UserMessage String message);  
}
```

```
# microprofile-config.properties  
dev.langchain4j.cdi.plugin.my-memory.class=\  
    dev.langchain4j.memory.chat.MessageWindowChatMemory  
dev.langchain4j.cdi.plugin.my-memory.config.max-messages=20
```

@MemoryId → une mémoire par session / utilisateur

RAG : RETRIEVAL AUGMENTED GENERATION

 Vos documents →  Embeddings →  Vector Store

 Question utilisateur →  Recherche →  Contexte →  LLM

```
@RegisterAIService(chatModelName = "my-model",
                    contentRetrieverName = "my-rag")
public interface KnowledgeAssistant {

    @SystemMessage("Answer based on the provided context")
    String answer(@UserMessage String question);
}
```

```
@ApplicationScoped
public class KnowledgeBaseProvider {
    @Inject ConferenceRepository repository;

    @Produces @Named("my-rag") @ApplicationScoped
    public ContentRetriever contentRetriever() {
        // EmbeddingModel + InMemoryEmbeddingStore + ingestion
        return EmbeddingStoreContentRetriever.builder()
            .embeddingStore(store)
            .embeddingModel(embeddingModel)
            .maxResults(3).build();
    }
}
```

TOOLS : FUNCTION CALLING

```
@ApplicationScoped
public class ExpeditionTools {

    @Inject
    ExpeditionService expeditionService;

    @Tool("List available Viking expeditions for a given date")
    public List<Expedition> listExpeditions(
        @P("Date in yyyy-MM-dd format") String date) {
        return expeditionService.getExpeditions(LocalDate.parse(date));
    }

    @Tool("Enroll a warrior in an expedition")
    public Enrollment enrollWarrior(
        @P("Warrior name") String name,
        @P("Desired date") String date) {
        return expeditionService.enroll(name, LocalDate.parse(date));
    }
}
```

Les Tools sont des **beans CDI** → injection de dépendances dans les tools ! 🎉



ÇA MARCHE EN LOCAL !

Mais en prod, le LLM a décidé
de répondre en Klingon.

Et votre endpoint timeout après 30 secondes.

On a besoin de **garde-fous** !

MICROPROFILE FAULT TOLERANCE

```
@RegisterAIService(chatModelName = "my-model")
public interface ResilientAssistant {

    @Retry(maxRetries = 3, delay = 1000)
    @Timeout(value = 30, unit = ChronoUnit.SECONDS)
    @Fallback(fallbackMethod = "fallbackChat")
    @CircuitBreaker(requestVolumeThreshold = 5,
        failureRatio = 0.5)
    String chat(@UserMessage String message);

    default String fallbackChat(String message) {
        return "Service temporarily unavailable. "
            + "Please try again in a moment.";
    }
}
```

MICROPROFILE TELEMETRY

TRACES

OpenTelemetry natif

-  Latence LLM
-  Tokens in/out
-  Appels tools
-  Chaîne RAG

MÉTRIQUES

Grafana LGTM

-  Coût par requête
-  Taux d'erreur
-  Temps de réponse
-  Circuit breaker

←!— Il suffit d'ajouter la dépendance →

```
<dependency>  
  <groupId>dev.langchain4j.cdi.mp</groupId>  
  <artifactId>langchain4j-cdi-telemetry</artifactId>  
</dependency>
```

CONFIGURATION OPENTELEMETRY

```
# MicroProfile OpenTelemetry - OTLP Exporter
otel.exporter.otlp.endpoint=http://localhost:4318
otel.service.name=demo-2-langchain4j-cdi
otel.traces.exporter=otlp
otel.metrics.exporter=otlp

# LangChain4j-CDI Telemetry - Listeners
dev.langchain4j.cdi.plugin.my-model.config.listeners=\
    dev.langchain4j.cdi.telemetry.SpanChatModelListener,\
    dev.langchain4j.cdi.telemetry.MetricsChatModelListener
```

Les listeners capturent automatiquement toutes les interactions IA

GRAFANA LGTM : UNE SEULE COMMANDE

```
# Démarrer le stack d'observabilité complet
podman run -p 3000:3000 -p 4317:4317 -p 4318:4318 \
  --rm -ti grafana/otel-lgtm
```

 **LGTM** = Loki + Grafana + Tempo + Mimir

- ✓ **Tempo** : stockage des traces
- ✓ **Loki** : agrégation des logs
- ✓ **Mimir** : métriques time-series
- ✓ **Grafana** : visualisation unifiée
- ✓ **OTLP Collector** : intégré sur les ports 4317/4318

Ouvrir <http://localhost:3000> → Explore → Tempo 



DEMO 2

L'ASSISTANT D'EXPÉDITION VIKING

Memory + RAG + Tools + Fault Tolerance + Telemetry



Mémoire par session (@MemoryId)



RAG sur les expéditions
(contentRetrieverName)



Tools d'inscription (@Tool + CDI)



Fault Tolerance via live coding !

LES MODULES LANGCHAIN4J-CDI

core

langchain4j-cdi

API + annotations

portable

langchain4j-cdi-portable-ext

WildFly, Payara, GlassFish, Liberty

+ LANGCHAIN4J-CDI-EL
EXPRESSION LANGUAGES

build

langchain4j-cdi-build-compatible-ext

Quarkus, Helidon

+ LANGCHAIN4J-CDI-A2A
AGENT-TO-AGENT PROTOCOL

résilience

langchain4j-cdi-fault-tolerance

Retry, Timeout, CircuitBreaker...

observabilité

langchain4j-cdi-telemetry

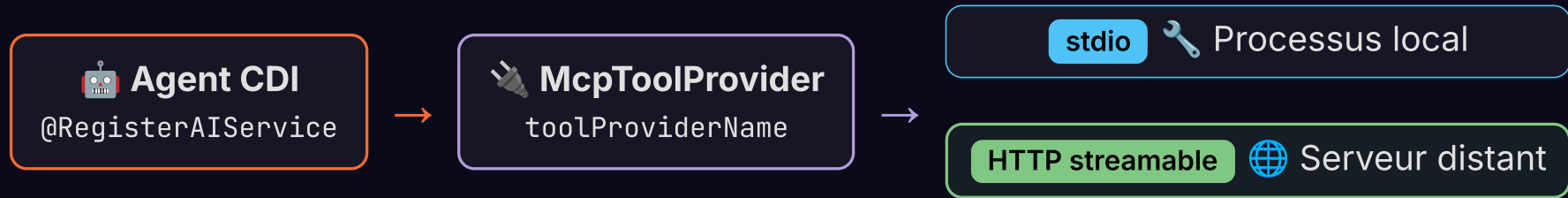
OpenTelemetry, métriques

bientôt ?

langchain4j-cdi-security

JWT, RBAC pour agents ?

MCP : MODEL CONTEXT PROTOCOL



```
@RegisterAIService(chatModelName = "my-model",
                    toolProviderName = "mcp-tools")
public interface AgentWithMCP {
    String execute(@UserMessage String task);
}
```

```
# MicroProfile Config → McpToolProvider via HTTP streamable
dev.langchain4j.cdi.plugin.mcp-tools.class=\
    dev.langchain4j.mcp.McpToolProvider
dev.langchain4j.cdi.plugin.mcp-tools.config.transport-type=http
dev.langchain4j.cdi.plugin.mcp-tools.config.url=\
    http://localhost:8081/mcp
```

Zéro code de plomberie -- tout passe par les properties 🌐



DEMO 3

HNEFATAFL AU GRAND THING AVEC MCP SUR WILDFLY

✂️ **Ragnar le Skald**, le Jarl qui anime le Hnefatafl au Grand Thing

✅ **Serveur MCP HTTP SSE** externe pour les lancers de dés

✅ Agent IA avec `@RegisterAIService` + `McpToolProvider`

✅ Configuration via le `subsystem AI WildFly`

✅ Tout configuré dans `standalone.xml`

Valhalla attend ! Hnefatafl sur WildFly ! ✂️ 🛡️

FLUX : LANCER DE RUNES VIA MCP SSE

1. **Guerrier** : "Lance les runes"
2. **Endpoint JAX-RS** → `HnefataflJarlAI.play()`
3. **LLM (Ollama)** via subsystem AI WildFly → décide d'utiliser `roll(numberOfDice=2)`
4. **McpToolProvider** → JSON-RPC → **Serveur MCP Runes** (HTTP SSE sur localhost:8090)
5. **Serveur MCP** lance 2d6 → retourne `{"rolls": [4, 3], "total": 7}` via SSE
6. **LLM** analyse le résultat → identifie "Faveur d'Odin" → répond au guerrier
7. **Ragnar le Skald** : "PAR ODIN! Faveur d'Odin ! Le guerrier triomphe !
 "

Le serveur MCP est accessible via socket-binding configuré dans standalone.xml 



LE SKALD A CHANTÉ EN ANGLAIS...

...avec des elfes *et* des nains dans la saga.

Le LLM est non-déterministe. Il fait ce qu'il veut.

La solution : **Guardrails** — les garde-fous de vos agents IA

GUARDRAILS : VALIDER L'ENTRÉE ET LA SORTIE

INPUTGUARDRAIL

Filtre les requêtes *avant* le LLM

- ✓ Langue valide ?
- ✓ Contenu approprié ?
- ✓ Pas de mots interdits ?

OUTPUTGUARDRAIL

Valide les réponses *après* le LLM

- ✓ Langue de la réponse ?
- ✓ Contenu conforme ?
- ✓ Format attendu ?

```
@RegisterAIService(  
    chatModelName      = "my-model",  
    inputGuardrailNames = {"french-input", "fantasy-input"},  
    outputGuardrailNames = {"french-output", "fantasy-output"})  
public interface SkaldAssistant {  
  
    @SystemMessage("Tu es un skald viking qui compose des chants épiques en FRANÇAIS...")  
    String composeSong(@UserMessage String request);  
}
```

GUARDRAILS = BEANS CDI NOMMÉS

```
@ApplicationScoped
@Named("french-input")    // ← Nom référencé dans @RegisterAIService
public class FrenchInputGuardrail implements InputGuardrail {

    private LanguageDetector detector;

    @PostConstruct
    void init() {
        detector = new OptimaizeLangDetector().loadModels();
    }

    @Override
    public InputGuardrailResult validate(UserMessage userMessage) {
        String text = userMessage.singleText();
        LanguageResult result;
        synchronized (detector) {          // ← Thread-safe !
            result = detector.detect(text);
        }
        if (result.isReasonablyCertain() && !"fr".equals(result.getLanguage())) {
            return failure("Seul le français est accepté !");
        }
        return success();
    }
}
```

Apache Tika pour la détection de langue — benefit of the doubt pour les textes courts ou ambigus



DEMO 4

SKALD VIKING AVEC GUARDRAILS

🛡️ **4 guardrails** autour d'un agent de composition de chants vikings

✅ **InputGuardrail** : langue française uniquement

✅ **InputGuardrail** : pas de races fantastiques dans la requête

✅ **OutputGuardrail** : le chant généré est bien en français

✅ **OutputGuardrail** : pas de nains ni d'elfes dans le chant

Les guardrails sont des beans CDI — injection de dépendances possible ! 🎉

OUTPUTGUARDRAIL : VALIDER LA RÉPONSE

```
@ApplicationScoped
@Named("fantasy-output")
public class NoFantasyRacesOutputGuardrail implements OutputGuardrail {

    private static final Set<String> FORBIDDEN_WORDS =
        Set.of("nain", "nains", "elfe", "elf", "elfes");

    @Override
    public OutputGuardrailResult validate(AiMessage responseFromLLM) {
        String text = responseFromLLM.text().toLowerCase();

        for (String word : FORBIDDEN_WORDS) {
            if (text.contains(word)) {
                return failure(
                    "Le chant du skald ne doit pas évoquer "
                    + "les nains ni les elfes ! C'est une saga viking pure.");
            }
        }
        return success();
    }
}
```

L'OutputGuardrail reçoit l'AiMessage complète — même pattern que l'InputGuardrail

A2A : AGENT-TO-AGENT PROTOCOL 🤝

MCP

Agent ↔ Outils

Un agent appelle des *outils* externes
(bases de données, APIs, fichiers...)

A2A

Agent ↔ Agent

Un agent **délègue** à d'autres agents
spécialisés, chacun sur son propre serveur



Les **microservices**, version agents IA

ARCHITECTURE A2A : STORY FORGE

Orchestrateur (port 8082)

LangChain4j Agentic : séquence + boucle + **Style Editor** (agent local)

Creative Writer

Agent A2A (port 8080)

@RegisterAIService + AgentCard

Style Scorer

Agent A2A (port 8081)

@RegisterAIService + AgentCard

 Skald forge la saga →  Juge évalue (score < 0.8 ?) →  Maître skald réécrit →  Boucle

SERVEUR A2A : CREATIVE WRITER

```
1 @RegisterAIService(chatModelName = "ollama", scope = ApplicationScoped.class)
2 public interface CreativeWriter {
3     @UserMessage("""
4         You are a Norse skald, a Viking bard who crafts mighty sagas.
5         Forge a short saga no more than 3 sentences around the given topic,
6         in the spirit of the Viking age.
7         Return only the saga in English and nothing else.
8         The topic is {{topic}}.
9         """)
10    String generateStory(@V("topic") String topic);
11 }
```

```
@Produces @PublicAgentCard
public AgentCard agentCard() {
    return AgentCard.builder()
        .name("Creative Writer")
        .description("Generate a story based on the given topic")
        .url(serverUrl)
        .capabilities(AgentCapabilities.builder()
            .streaming(true).build())
        .skills(List.of(AgentSkill.builder()
            .id("creative_writer").name("Creative Writer")
            .description("Generate a story based on the given topic")
            .build()))
        .preferredTransport(TransportProtocol.JSONRPC.asString())
        .build();
}
```

@RegisterAIService classique + **@PublicAgentCard** producteur CDI = agent exposé sur le réseau A2A

AGENTEXECUTOR : LE PONT A2A ↔ CDI

```
1 @Inject
2 CreativeWriter creativeWriter;
3
4 @Produces
5 public AgentExecutor agentExecutor() {
6     return new AgentExecutor() {
7         @Override
8         public void execute(RequestContext context, AgentEmitter emitter)
9             throws A2AError {
10             emitter.startWork();
11
12             String userMessage = extractTextFromMessage(
13                 context.getMessage());
14             String response = creativeWriter.generateStory(userMessage);
15
16             emitter.addArtifact(
17                 List.of(new TextPart(response)),
18                 null, null, null);
19             emitter.complete();
20         }
21     };
22 }
```

AgentExecutor reçoit la requête A2A → appelle le **@RegisterAIService** → renvoie la réponse comme artifact

ORCHESTRATEUR : DÉCLARER LES AGENTS

```
1 @RegisterA2AAgent(name = "creative-writer",
2     a2aServerUrl = "${a2a.creative-writer.url}",
3     outputKey = "story",
4     description = "Generate a Norse saga based on the given topic")
5 public interface A2ACreativeWriter {
6     String generateStory(@V("topic") String topic);
7 }
8
9 @RegisterA2AAgent(name = "style-scorer",
10     a2aServerUrl = "${a2a.style-scorer.url}",
11     outputKey = "score",
12     description = "Score a saga based on how well it captures a given style")
13 public interface A2AStyleScorer {
14     double scoreStyle(@V("story") String story, @V("style") String style);
15 }
16
17 @RegisterSimpleAgent(name = "style-editor",
18     description = "Reforge a saga to better capture a given style",
19     outputKey = "story", chatModelName = "ollama")
20 public interface StyleEditor {
21     @UserMessage("""
22         You are a master Norse skald who shapes and tempers sagas
23         like a smith forges steel. Rewrite the following saga to better
24         honor and embody the {{style}} style.
25         Return only the saga and nothing else.
26         The saga is "{{story}}")
```

@RegisterA2AAgent → proxy vers agent distant | **@RegisterSimpleAgent** → agent local avec LLM

ORCHESTRATEUR : SÉQUENCE + BOUCLE

```
1 @RegisterSequenceAgent(  
2     subAgentNames = {"creative-writer", "style-review-loop"},  
3     outputKey = "story")  
4 public interface StyledWriter extends AgenticScopeAccess {  
5     ResultWithAgenticScope<String> writeStoryWithStyle(  
6         @V("topic") String topic, @V("style") String style);  
7 }  
8  
9 @RegisterLoopAgent(name = "style-review-loop",  
10     subAgentNames = {"style-scorer", "style-editor"},  
11     maxIterations = 5,  
12     exitConditionName = "scoreExitCondition",  
13     exitConditionDescription = "Exit when the story score reaches 0.8 or higher")  
14 public interface StyleReviewLoop {  
15     String refineStory(@V("story") String story, @V("style") String style);  
16 }  
17  
18 @ApplicationScoped  
19 @Named("scoreExitCondition")  
20 public class ScoreExitCondition implements Predicate<AgenticScope> {  
21     @Override  
22     public boolean test(AgenticScope scope) {  
23         Object raw = scope.readState("score");  
24         if (raw instanceof Number n) {  
25             return n.doubleValue() ≥ 0.8;  
26         }
```

@RegisterSequenceAgent → pipeline séquentiel | @RegisterLoopAgent → boucle déclarative | @Named → condition de sortie
CDI



DEMO 5

STORY FORGE — ORCHESTRATION A2A

🤝 3 services WildFly communiquant via le protocole A2A

✓ **Creative Writer** : forge une saga nordique sur un sujet

✓ **Style Scorer** : juge l'alignement de la saga avec un style

✓ **Orchestrateur** : séquence + boucle de révision (score ≥ 0.8)

✓ **Style Editor** : réécriture locale si le score est insuffisant

Les microservices IA, câblés avec CDI et LangChain4j Agentic 🚀

CE QU'ON A VU AUJOURD'HUI

 Une **extension CDI** qui transforme les interfaces en agents IA

 **@Inject** un agent aussi simplement qu'un EntityManager

 Configuration 100% **MicroProfile Config**

 Memory, RAG, Tools, **MCP** -- tout injectable

 **Fault Tolerance** + **Telemetry** out of the box

 **Guardrails** -- valider entrée et sortie avec des beans CDI

 **A2A** -- orchestration multi-agents avec LangChain4j Agentic

 Compatible avec **tous** les serveurs CDI

REJOIGNEZ-NOUS !

🌟 github.com/langchain4j/langchain4j-cdi

🐛 Issues bienvenues -- même les demandes de fonctionnalités folles

🔗 PRs encouragées -- on review vite (promis)

💬 GitHub Discussions -- pour les questions

Merci aussi à [Buhake Sindi](#), co-créateur 🙏



QUESTIONS ?

"L'IA va-t-elle remplacer les développeurs Java ?"

Non. Mais un développeur Java **avec** l'IA
remplacera celui sans.

Et avec CDI, l'IA c'est juste un @Inject de plus. 😊

MERCI ! 🙏

Yann Blazart — @yblazart

Emmanuel Hugonnet — @ehugonnet

🌟 github.com/langchain4j/langchain4j-cdi

RivieraDev 2026

Slides : Reveal.js | D mos : WildFly + IntelliJ

SLIDES & CODE



github.com/ehsavoie/langchain4j-cdi-lab-2026